



MudRYder - Personnage

PROJET FINAL - MUDRYDER

Gabriel Zeizer, Aurélien Santi

10 juin 2026

Table des matières

1 – Introduction	3
2 – Moyens à disposition	3
2.1 – Github	3
2.2 – GDX2D	3
3 – Inspiration	4
4 – Choix d'implémentation	4
4.1 – Architecture générale	4
4.2 – Gestion des modes de jeu	5
4.3 – Deux systèmes de dessin distincts	5
4.4 – Hiérarchie des lignes	5
4.5 – Gestion de la caméra	5
4.6 – Optimisation du rendu	6
4.7 – Système de sauvegarde et chargement	6
4.8 – Détection de collision pour l'effacement	6
4.9 – Gestion audio	7
5 – Problèmes rencontrés et solutions	7
5.1 – Surcharge CPU	7
5.2 – Crash avec des tableaux de lignes vides	7
5.3 – Gestion des angles du personnage	7
5.4 – Optimisation des calculs vectoriels	7
6 – Fonctionnalités	7
6.1 – Ce qui fonctionne	7
6.2 – Limites connues	7
7 – Améliorations et auto-critique	8
7.1 – Propositions d'améliorations	8
7.2 – Auto-critique	8
8 – Conclusion	8
9 – Annexes	8
9.1 – Point d'entrée	8
9.2 – Cœur du jeu	8
9.3 – Systèmes de dessin	15
9.4 – Types de lignes	21
9.5 – Fenêtres UI	22
9.6 – Modes de jeu	25
9.7 – Types	27
Références	28

1 – Introduction

Ce projet a été effectué dans le cadre du cours de Programmation Orientée-Objets. Il s'agissait d'un projet plutôt libre dont l'objectif était de créer un jeu vidéo afin de mettre en oeuvre les concepts acquis tout au long de l'année.

Nous avons choisi de recréer le jeu **Line Rider**, un classique du genre « bac à sable » où le joueur dessine des pistes qu'un personnage parcourt ensuite par gravité.

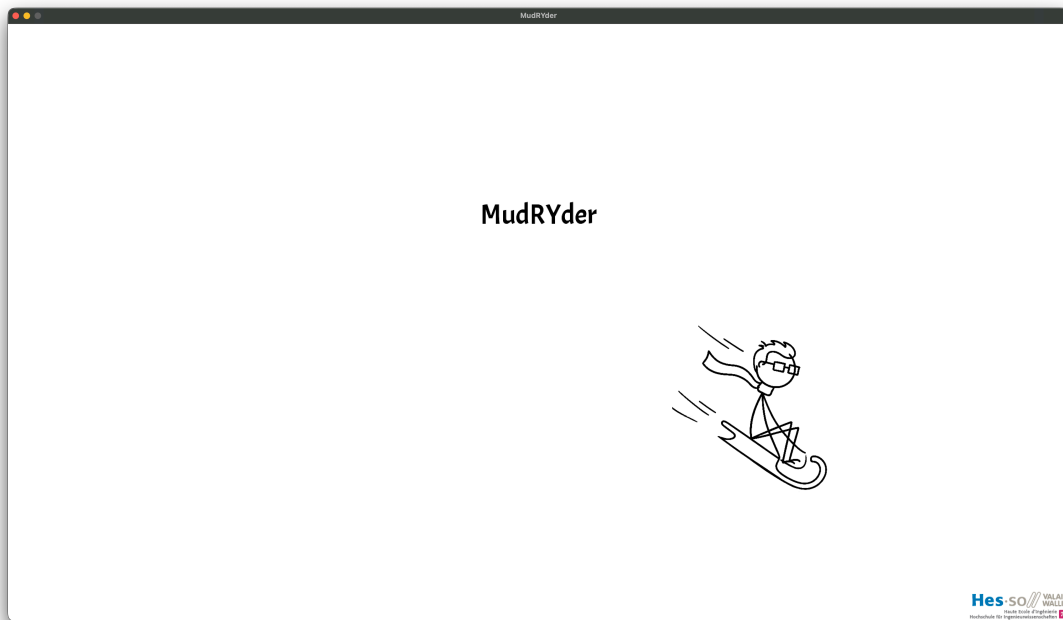


Figure 1 - Écran de démarrage du jeu

2 – Moyens à disposition

Afin d'assurer le bon déroulement de ce projet, nous avons utilisé la librairie GDX2, une bibliothèque Java simple, sur laquelle nous nous sommes appuyés afin de gérer la partie graphique et physique. Le projet a été entièrement écrit en Scala 2.13.18 à l'aide d'IntelliJ IDEA.



Figure 3 - Logo de Scala



Figure 4 - Logo de IntelliJ IDEA

2.1 – Github

Ce projet étant commun, l'utilisation de Git nous a permis de collaborer de manière efficace et rapide. Nous avons utilisé Github pour sa simplicité d'intégration dans notre processus de développement.

2.2 – GDX2D

La bibliothèque GDX2D est un projet open-source créée par divers enseignants de l'HES-SO Valais/Wallis. Le projet a été basé sur une bibliothèque déjà existante : libGDX [1].

3 – Inspiration

Nous avons eu l'idée de nous lancer dans la création de ce jeu en nous inspirant de Line Rider [2], un jeu de style bac à sable dans lequel un rider glisse le long d'une ligne dessinée par le joueur. Ce type de jeu est très différent de la majorité : il n'a pas de gagnant ou de perdant, mais il en va de la créativité de chacun.

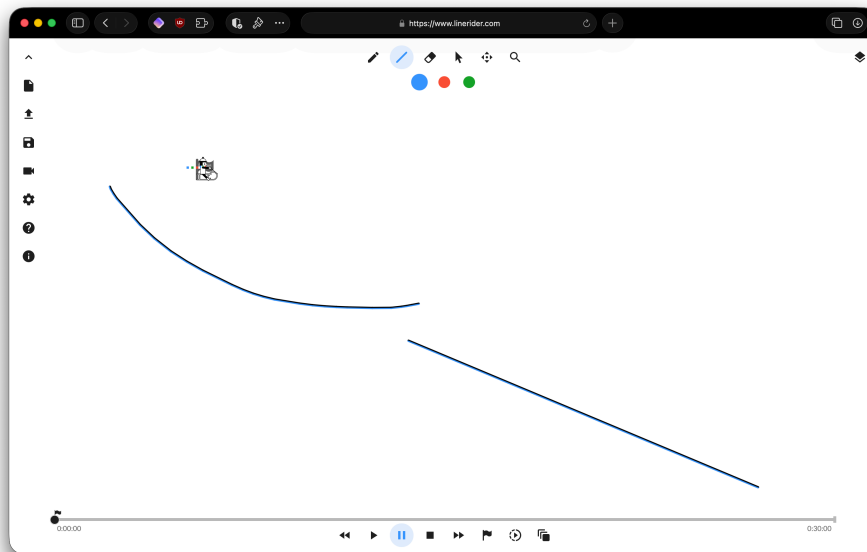


Figure 5 - Line Rider, le jeu original

4 – Choix d'implémentation

4.1 – Architecture générale

Le projet est structuré autour d'une classe principale `Game` qui étend `DesktopApplication` de la bibliothèque GDX2D. Cette classe gère la boucle de jeu principale et délègue les différentes fonctionnalités à des sous-systèmes spécialisés : `LineDrawMachine` et `FreeDrawMachine` pour le dessin, `MenuModesMachine` pour la gestion des modes, `MudryMachine` pour le personnage, et `MusicPlayer` pour l'audio.

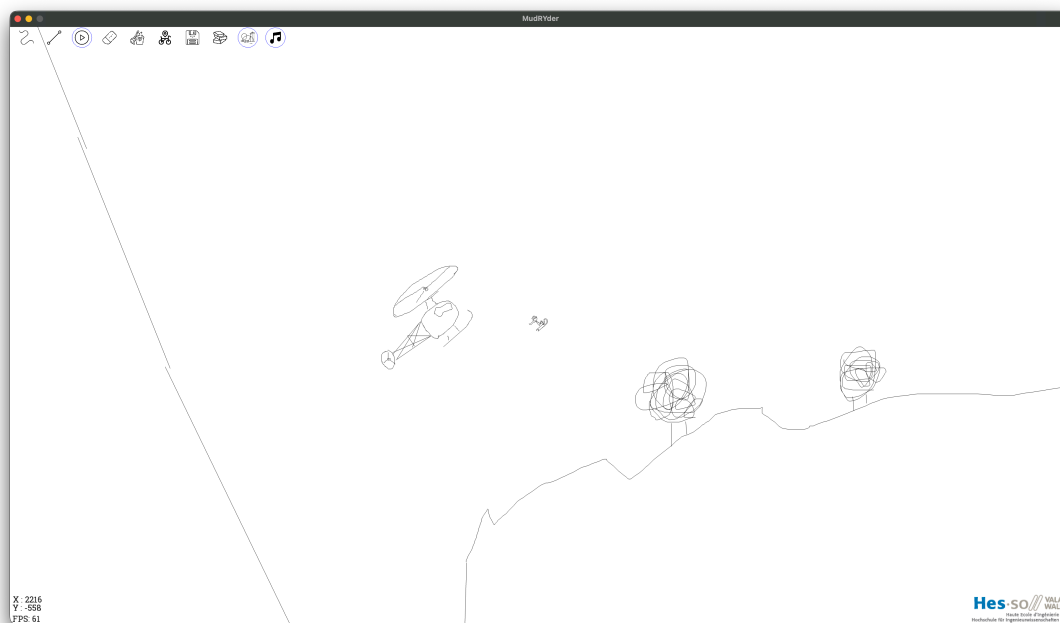


Figure 6 - Mudry en pleine course sur les pistes

4.2 – Gestion des modes de jeu

Nous avons implémenté une machine à états simple via la classe `MenuModesMachine`. Celle-ci gère les modes (`lines`, `free`, `play`, `eraser`, `mop`, `save`, `load`) ainsi que des toggles internes pour le mode de dessin (`physic` / `decoration`) et la musique (`music` / `musicmute`). Le mode courant est stocké sous forme de chaîne de caractères et transmis aux différentes machines de dessin, qui adaptent leur comportement en conséquence. Par exemple, en mode `play`, le dessin est désactivé et la caméra suit automatiquement le personnage.



Figure 7 - Menu des modes de jeu

4.3 – Deux systèmes de dessin distincts

Nous avons fait le choix de séparer le dessin de lignes segmentées (`LineDrawMachine`) et le dessin de lignes libres (`FreeDrawMachine`). La première fonctionne par clic, glisser et relâcher pour créer un segment rectiligne entre deux points, tandis que la seconde génère des traits continus en ajoutant des segments à chaque frame de glissement. Cette séparation s'explique par la différence fondamentale d'interaction utilisateur et de structure de données : `LineArray` pour les segments fixes, `FreeArray` (tableau de tableaux) pour les traits libres.

4.4 – Hiérarchie des lignes

Nous avons défini un trait `Line` comme interface commune à toutes les lignes, imposant les propriétés `p1x`, `p1y`, `p2x`, `p2y` et les méthodes `draw` et `destroy`. Deux implémentations concrètes en `case class` en découlent :

- `PhysicLine` étend `PhysicsStaticLine` (corps physique `Box2D` [3]) et mixe le trait `Line`
- `DecoLine` implémente uniquement le trait `Line` sans composante physique

L'utilisation de `case class` nous permet de bénéficier de l'immuabilité des coordonnées, du pattern matching, et de l'égalité structurelle. L'héritage multiple de `PhysicLine` (classe physique + trait) a nécessité l'utilisation de `super[PhysicsStaticLine].destroy()` pour lever l'ambiguïté.

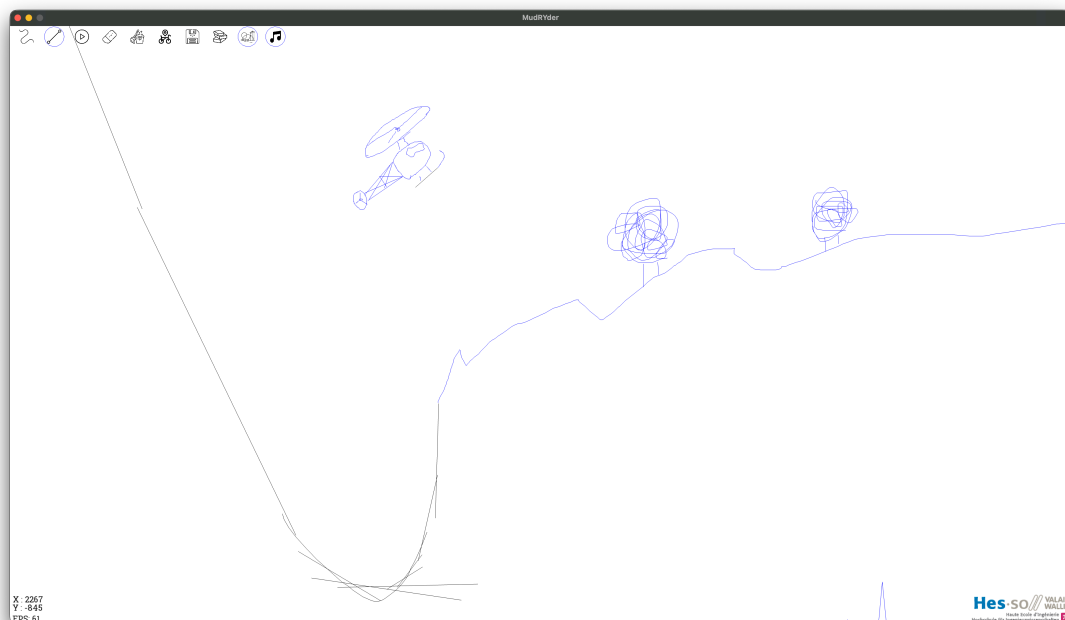


Figure 8 - Lignes physiques (noir) et décoratives (bleu)

4.5 – Gestion de la caméra

La caméra fonctionne selon deux modes. En mode édition (dessin), l'utilisateur peut se déplacer librement dans le monde via un glisser-déposer au clic droit, ce qui décale les coordonnées de la caméra (`camX`,

camY). En mode `play`, la caméra suit automatiquement la position du personnage (`playerMachine.posX`, `playerMachine.posY`). Les coordonnées de clic souris sont converties en coordonnées monde via la formule $x + (\text{camX} - 960)$ pour l'axe X, ce qui permet d'interagir correctement avec les éléments même après un déplacement de la vue.

4.6 – Optimisation du rendu

Afin de maintenir des performances élevées, seules les lignes situées dans un rayon de 2000 pixels autour de la caméra sont dessinées. Cela nous a permis de réduire l'impact sur la performance plus le nombre de lignes augmente.

4.7 – Système de sauvegarde et chargement

Le format de sauvegarde choisi est le CSV, simple à lire et à écrire. Chaque ligne est enregistrée au format `TypeLigne,x1,y1,x2,y2` dans le dossier `./saves/`. Le nom de fichier est généré automatiquement à partir de la date et de l'heure (`save_JJJHHMMSS.csv`). Au chargement, le fichier est parsé et chaque ligne est reconstruite en `PhysicLine` ou `DecoLine` selon son type. Ce format texte présente l'avantage d'être lisible par un humain et facile à déboguer.

Voici un exemple de fichier de sauvegarde :

```
1 PhysicLine,100.0,200.0,500.0,200.0
2 DecoLine,300.0,400.0,600.0,400.0
3 PhysicLine,500.0,200.0,800.0,600.0
```

Listing 1 - Exemple de fichier de sauvegarde au format CSV

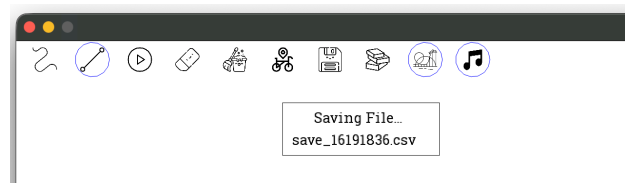


Figure 9 - Notification de sauvegarde

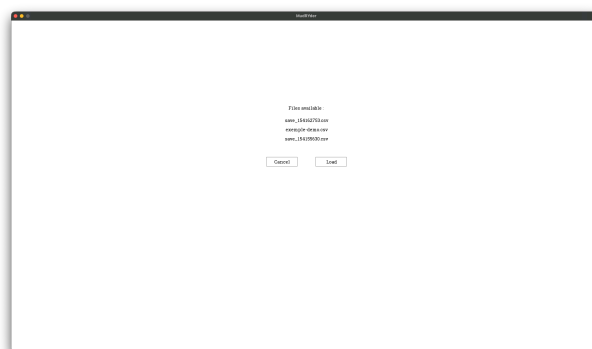


Figure 10 - Fenêtre de chargement

Chaque ligne contient : `TypeLigne`, `x1`, `y1`, `x2`, `y2`. Le type `PhysicLine` crée une ligne avec collision physique, `DecoLine` une ligne purement décorative (bleue, sans physique).

4.8 – Détection de collision pour l'effacement

La classe `Calculator` implémente un algorithme de distance point-segment basé sur le produit vectoriel. Cette méthode permet de déterminer si un clic est suffisamment proche d'une ligne (tolérance de 8 pixels) pour l'effacer. L'outil « gomme » parcourt toutes les lignes et supprime celles qui se trouvent dans le rayon du curseur, tandis que l'outil « serpillière » (`mop`) efface l'intégralité du dessin après confirmation de l'utilisateur via la fenêtre `AreYouSureWindow`.

4.9 – Gestion audio

Le `MusicPlayer` utilise l'API `javax.sound.sampled.AudioSystem` pour lire des fichiers WAV. Deux pistes distinctes sont utilisées : une pour le mode édition (`edit.wav`, en boucle) et une pour le mode jeu (`play.wav`, lecture unique). Le son peut être coupé via un bouton dans le menu, ce qui stoppe et ferme le clip audio pour libérer les ressources.

5 – Problèmes rencontrés et solutions

5.1 – Surcharge CPU

Au fur et à mesure que le nombre de lignes augmentait, le jeu commençait à ralentir considérablement. Le problème venait du fait que toutes les lignes étaient dessinées à chaque frame, même celles situées loin de la caméra. Nous avons résolu ce problème en implémentant un système où seules les lignes situées dans un rayon de 2000 pixels autour de la caméra sont dessinées. Cela a considérablement amélioré les performances.

5.2 – Crash avec des tableaux de lignes vides

Un crash survenait lorsque le joueur effaçait toutes les lignes puis tentait d'en dessiner une nouvelle. Le moteur physique ne gère pas correctement un monde vide. Nous avons contourné le problème en insérant une ligne factice très loin de l'écran (coordonnées `-10000`, `-10000`) dès que le tableau est vide (`ArrayEmptyFix`).

5.3 – Gestion des angles du personnage

Initialement, le personnage ne suivait pas correctement l'inclinaison des pentes. Nous avons modifié la méthode `collision()` de `MudryMachine` pour lire l'angle du segment contacté et appliquer une correction au-delà de ± 120 degrés, stabilisant ainsi le comportement.

5.4 – Optimisation des calculs vectoriels

L'outil gomme utilisait des calculs de distance point-segment coûteux. Nous avons optimisé l'algorithme en ajoutant une vérification préalable des limites avant le calcul du produit vectoriel. Nous avons également essayé de réduire le nombre de cycles CPU nécessaires en modifiant certains calculs. En remplaçant un `math.pow(x,2)` par simplement `x * x` de cette manière, nous avons encore pu réduire l'impact des calculs.

6 – Fonctionnalités

6.1 – Ce qui fonctionne

- Dessin de lignes (segmentées et libres, physiques et décoratives)
- Physique complète (gravité, collisions avec le personnage)
- Caméra avec deux modes (suivi automatique en jeu, libre en édition)
- Outils d'effacement (gomme individuelle et serpillière avec confirmation)
- Sauvegarde et chargement de pistes au format CSV
- Écran de démarrage animé, musique d'ambiance, HUD (FPS, coordonnées)

6.2 – Limites connues

- La musique peut recommencer lors de certains événements, par exemple lors de l'exploration des fichiers enregistrés.
- La fenêtre de chargement suppose que le dossier `saves/` contient au moins un fichier valide
- Si un fichier est généré / ajouté au dossier `saves/` après avoir lancé le jeu, il n'apparaîtra pas.
- Le personnage ne suit pas parfaitement l'angle de la pente

7 – Améliorations et auto-critique

7.1 – Propositions d’améliorations

- Ajouter un outil « accélérateur » pour propulser Mudry (prévu dans la roadmap initiale)
- Rendre le déplacement sur les angles plus fluide
- Unifier les deux systèmes de dessin (`LineDrawMachine` et `FreeDrawMachine`) pour réduire la duplication de code
- Ajouter une gestion d’erreur (fichiers, index, dossier vide)
- Remplacer le correctif `ArrayEmptyFix` par une solution plus propre
- Améliorer la structure du code pour le rendre plus simple à consulter et modifier

7.2 – Auto-critique

Le projet s’est bien déroulé dans l’ensemble. L’utilisation de Git nous a permis de travailler en parallèle efficacement. Cependant, le découpage en classes aurait pu être mieux pensé en amont : la séparation en deux systèmes de dessin distincts a créé de la duplication et des incohérences, notamment pour la gomme et la sauvegarde. Nous avons passé beaucoup de temps à implémenter des fonctionnalités mais pas assez à restructurer le code et à l’optimiser. À l’avenir, il faudrait prioriser la robustesse du code avant les fonctionnalités secondaires.

8 – Conclusion

Ce projet nous a permis de mettre en pratique les concepts de programmation orientée-objet vus durant le semestre : héritage, polymorphisme, traits, *case classes* et *pattern matching*. Malgré quelques limitations, le jeu est fonctionnel et atteint les objectifs fixés. Nous sommes satisfaits du résultat et cette expérience nous a appris l’importance d’une bonne architecture logicielle ainsi que d’une répartition claire des tâches au sein d’une équipe.

9 – Annexes

Le code source complet du projet est présenté ci-dessous.

9.1 – Point d’entrée

```

1 object main {
2   def main(args: Array[String]): Unit = {
3     new Game().launch()
4   }
5 }
```

Listing 2 - Point d’entrée du programme - main.scala

9.2 – Cœur du jeu

```

1 import ch.hevs.gdx2d.desktop.DesktopApplication
2 import ch.hevs.gdx2d.lib.physics.PhysicsWorld
3 import ch.hevs.gdx2d.lib.{GdxGraphics, ScreenManager}
4 import com.badlogic.gdx.Input
5 import com.badlogic.gdx.graphics.{Color, OrthographicCamera}
6 import com.badlogic.gdx.math.Vector2
7
8 import java.util.Calendar
9
10 class Game extends DesktopApplication(1920, 1080) {
11   val lineMachine = new LineDrawMachine
12   val modesMachine = new MenuModesMachine
13   val freeMachine = new FreeDrawMachine
14   val uSure = new AreYouSureWindow
15   val saveWin = new SavingWindow
16   val walkman = new MusicPlayer
```



```

17 val s = new ScreenManager
18 val s1 = new ScreenManager
19 val startTime = System.currentTimeMillis()
20 val l = PhysicLine(0, 100, 1920, 100)
21 val l2 = PhysicLine(0, 1000, 1920, 100)
22 var playerMachine: MudryMachine = _
23 var onMenuClick: Boolean = false
24 var iAmClicked: Boolean = false
25 var currentMode = ""
26 var musicMode = "music"
27 var lastMouseClicked = 1
28 var cam = new OrthographicCamera
29 var camX: Int = 1920 / 2
30 var camY: Int = 540
31 var RDragX: Int = 0
32 var RDragY: Int = 0
33 var lastMode = ""
34 var currentX: Int = 0
35 var currentY: Int = 0
36 var stableXOffset: Int = 0
37 var stableYOffset: Int = 0
38 var wasMopping: Boolean = false
39 var drawMode: String = "physic"
40 var firstRun = true
41 var isSaving = false
42 var savefile = ""
43 var saveTime: Long = _
44 var drawZoneX : Int = 0
45 var drawZoneY : Int = 0
46
47 override def onInit(): Unit = {
48     playerMachine = new MudryMachine("rider", new Vector2(960, 900), 4f, 0f, 0f, 0.00001f)
49     camX = playerMachine.posX.toInt
50     camY = playerMachine.posY.toInt
51     setTitle("MudRYder")
52     modesMachine.loadIcons()
53     s.registerScreen(classOf[SplashScreenWindow])
54     s1.registerScreen(classOf[LoadWindow])
55 }
56
57 override def onGraphicRender(g: GdxGraphics): Unit = {
58     walkman.play(currentMode, musicMode)
59     if (System.currentTimeMillis() - startTime < 3000) {
60         s.render(g)
61     } else {
62         if (firstRun) {
63             l.destroy()
64             l2.destroy()
65             playerMachine.setPos(camX, camY)
66         }
67         if (currentMode == "load") {
68             camX = 1920 / 2
69             camY = 540
70             cam.position.set(camX, camY, 0)
71             cam.update()
72             s1.render(g)
73         } else {
74             firstRun = false
75             g.clear(Color.WHITE)
76             g.setColor(Color.BLACK)
77
78             cam = g.getCamera
79             PhysicsWorld.updatePhysics()
80             playerMachine.update()
81
82             if (currentMode == "play") {
83                 cam.position.set(playerMachine.posX, playerMachine.posY, 0)
84             } else {
85                 cam.position.set(camX, camY, 0)
86             }
87         }
88     }
89 }

```

```

87     cam.update()
88
89     playerMachine.draw(g)
90
91     if (currentMode != "play") {
92         playerMachine.sleep()
93         playerMachine.angle = 0
94     }
95
96     if(currentMode != "play") drawZoneX = camX else drawZoneX = playerMachine.posX.toInt
97     if(currentMode != "play") drawZoneY = camY else drawZoneY = playerMachine.posY.toInt
98
99     lineMachine.drawLines(g, modesMachine.currentMode(), modesMachine.getDrawMode(),drawZoneX,drawZoneY)
100    freeMachine.drawFreeLines(g, modesMachine.currentMode(), modesMachine.getDrawMode(),drawZoneX,drawZoneY)
101
102    lastMode = currentMode
103    currentMode = modesMachine.currentMode()
104    musicMode = modesMachine.getMusicMode()
105
106
107    g.resetCamera()
108    modesMachine.drawModesMenu(g, g.getScreenHeight, 0)
109    if (currentMode == "mop") {
110        uSure.drawWindow(g)
111    }
112    if (isSaving && currentMode != "mop" &&
113        (System.currentTimeMillis() - saveTime) < 5000) {
114        saveWin.drawWindow(g, savefile)
115    }
116
117    val oldColor = g.sbGetColor()
118    g.setColor(Color.BLACK)
119    if (currentMode == "play") {
120        g.drawString(5, 50, s"X : ${playerMachine.posX.toInt}")
121        g.drawString(5, 35, s"Y : ${playerMachine.posY.toInt}")
122    } else {
123        g.drawString(5, 50, s"X : ${camX}")
124        g.drawString(5, 35, s"Y : ${camY}")
125    }
126    g.setColor(oldColor)
127    g.drawFPS(Color.BLACK)
128    g.drawSchoolLogo()
129    }
130    }
131    }
132
133    override def onClick(x: Int, y: Int, button: Int): Unit = {
134        currentX = x
135        currentY = y
136
137        lastMouseClicked = button //On enregistre si click droit ou gache pour prevent le drag sur clic droit
138
139        super.onClick(x, y, button)
140
141        if (button == Input.Buttons.LEFT) {
142            onMenuClick = modesMachine.onMenuClick(x + stableXOffset, y - stableYOffset)
143            currentMode = modesMachine.currentMode()
144
145            //Maintenant que le monde bouge il faut calculer différemment l'emplacement de la souris
146            // x et y du clic sont relatif à la fenêtre et non au monde, le pixel haut gauche sera toujours à (0,0)
147            // indépendamment du mouvement dans le monde derrière
148            // il faut donc calculer par nous même cette position
149
150            val inWorldClicX: Int = x + (camX - 960)
151            val inWorldClicY: Int = y + (camY - 540)
152
153            currentMode match {
154                case "free" => {
155                    if (!onMenuClick) {
156                        freeMachine.onClick("LEFT", inWorldClicX, inWorldClicY)
157                    } else {

```

```

158     playerMachine.setPos(960, 900)
159 }
160 }
161 case "lines" => {
162     if (!onMenuClick) {
163         lineMachine.onClick("LEFT", inWorldClicX, inWorldClicY)
164     } else {
165         playerMachine.setPos(960, 900)
166     }
167 }
168 case "play" => {
169     if (lastMode != "play") {
170         playerMachine.setPos(960, 900)
171         playerMachine.awake()
172     }
173 }
174 case "eraser" => {
175     playerMachine.setPos(960, 900)
176     iAmClicked = true
177     freeMachine.clean(inWorldClicX, inWorldClicY)
178     lineMachine.clean(inWorldClicX, inWorldClicY)
179 }
180 case "mop" => {
181     uSure.onClick(x, y)
182     wasMopping = true
183     if (uSure.getAnswer() == "yes") {
184         freeMachine.mop()
185         lineMachine.mop()
186         modesMachine.modeSwitcher("lines")
187         lineMachine.onClick("LEFT", -100000, -100000)
188         lineMachine.endPoint.set(0f, 0f)
189         lineMachine.startPoint.set(0f, 0f)
190     } else if (uSure.getAnswer() == "no") {
191         modesMachine.modeSwitcher("lines")
192         lineMachine.onClick("LEFT", -100000, -100000)
193         lineMachine.endPoint.set(0f, 0f)
194         lineMachine.startPoint.set(0f, 0f)
195     }
196 }
197 case "return" => {
198     camX = playerMachine.posX.toInt
199     camY = playerMachine.posY.toInt
200     modesMachine.modeSwitcher("lines")
201 }
202 case "save" => {
203     isSaving = true
204     val c: Calendar = Calendar.getInstance()
205     val fn: String = "save_" +
206         s"${c.get(Calendar.DAY_OF_YEAR)}" +
207         s"${c.get(Calendar.HOUR_OF_DAY)}" +
208         s"${c.get(Calendar.MINUTE)}" +
209         s"${c.get(Calendar.SECOND)}"
210
211     lineMachine.save(fn)
212     freeMachine.save(fn)
213     savefile = fn + ".csv"
214     saveTime = System.currentTimeMillis()
215     println(s"Game saved : $fn.csv")
216     modesMachine.modeSwitcher("lines")
217 }
218 case "load" => {
219     s1.getActiveScreen match {
220         case lw: LoadWindow => {
221             lw.onClick(x, y)
222             //println(lw.getAnswer())
223             if (lw.getAnswer() == "load") {
224                 lineMachine.mop()
225                 freeMachine.mop()
226                 lineMachine.load(lw.getSelection())

```

```

227     modesMachine.modeSwitcher("lines")
228     lineMachine.onClick("LEFT", -100000, -100000)
229     lineMachine.endPoint.set(0f, 0f)
230     lineMachine.startPoint.set(0f, 0f)
231 }
232 if (lw.getAnswer() == "cancel") {
233     modesMachine.modeSwitcher("lines")
234     lineMachine.onClick("LEFT", -100000, -100000)
235     lineMachine.endPoint.set(0f, 0f)
236     lineMachine.startPoint.set(0f, 0f)
237 }
238 }
239 case _ =>
240 }
241 }
242 case _ => {
243     playerMachine.setPos(960, 900)
244 }
245 }
246
247 } else {
248     RDragX = x
249     RDragY = y
250     if (currentMode == "play") {
251         modesMachine.modeSwitcher("lines")
252         playerMachine.setPos(960, 900)
253         camX = playerMachine.posX.toInt
254         camY = playerMachine.posY.toInt
255     }
256 }
257
258 }
259
260
261 override def onDrag(x: Int, y: Int): Unit = {
262     currentX = x
263     currentY = y
264     val physicMode: String = modesMachine.getDrawMode()
265
266     val inWorldClicX: Int = x + (camX - 960)
267     val inWorldClicY: Int = y + (camY - 540)
268
269     if (!onMenuClick && lastMouseClicked == Input.Buttons.LEFT) {
270         currentMode match {
271             case "free" => freeMachine.onDrag(inWorldClicX, inWorldClicY, physicMode)
272             case "lines" => if (!wasMopping) lineMachine.onDrag(inWorldClicX, inWorldClicY)
273             case "play" => {}
274             case "eraser" => {
275                 freeMachine.clean(inWorldClicX, inWorldClicY)
276                 lineMachine.clean(inWorldClicX, inWorldClicY)
277             }
278             case _ => {}
279         }
280     } else if (lastMouseClicked == Input.Buttons.RIGHT && currentMode != "load") { // si drag sur clic droit
281         camX = camX - (x - RDragX)
282         camY = camY - (y - RDragY)
283         RDragX = x
284         RDragY = y
285         cam.position.set(camX, camY, 0)
286         cam.update()
287     }
288 }
289
290
291 override def onRelease(x: Int, y: Int, button: Int): Unit = {
292     currentX = x
293     currentY = y
294     val physicMode: String = modesMachine.getDrawMode()
295
296     super.onRelease(x, y, button)

```

```

297
298 val inWorldClicX: Int = x + (camX - 960)
299 val inWorldClicY: Int = y + (camY - 540)
300
301 if (!onMenuClick) {
302     super.onRelease(x, y, button)
303     lineMachine firstRun = false
304     freeMachine firstRun = false
305
306     if (button == Input.Buttons.LEFT) {
307         currentMode match {
308             case "free" => freeMachine.onRelease("LEFT", inWorldClicX, inWorldClicY)
309             case "lines" => if (!wasMopping) {
310                 lineMachine.onRelease("LEFT", inWorldClicX, inWorldClicY, physicMode)
311             } else {
312                 wasMopping = false
313             }
314             case "play" => {}
315             case "craser" => iAmClicked = false
316             case _ => {}
317         }
318         //println("Left button released")
319     } else {
320         //println("Right button released")
321     }
322 }
323 }
324 }

```

Listing 3 - Classe principale du jeu - Game.scala

```

1 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2 import ch.hevs.gdx2d.components.physics.primitives.PhysicsCircle
3 import ch.hevs.gdx2d.components.physics.utils.PhysicsConstants
4 import ch.hevs.gdx2d.lib.GdxGraphics
5 import ch.hevs.gdx2d.lib.interfaces.DrawableObject
6 import ch.hevs.gdx2d.lib.physics.AbstractPhysicsObject
7 import com.badlogic.gdx.math.Vector2
8
9 class MudryMachine(name: String, position: Vector2, radius: Float, density: Float, restitution: Float, friction: Float)
10 extends PhysicsCircle(name, position, radius, density, restitution, friction) with DrawableObject {
11
12     var firstRun = true
13     var posX: Float = 960
14     var posY: Float = 900
15     var angle: Float = 0
16     var img: BitmapImage = _
17     var rider: BitmapImage = _
18     var imgDown: BitmapImage = _
19     var selectedImage: BitmapImage = _
20     private var lastCollision = 0.5f
21
22     def setPos(x: Int, y: Int): Unit = {
23         val muBody = this.getBody
24         val positionMeters = new Vector2(x, y).scl(PhysicsConstants.P2M)
25         muBody.setTransform(positionMeters, 0)
26         this.setBodyLinearVelocity(new Vector2(0, 0))
27         posX = x
28         posY = y
29     }
30
31     def awake(): Unit = {
32         this.setBodyAwake(true)
33         this.enableCollisionListener()
34     }
35
36     def sleep(): Unit = {
37         this.setBodyAwake(false)
38     }

```

```

39
40 def draw(g: GdxGraphics): Unit = {
41     loadImages()
42
43     if (math.abs(this.angle) > 30) {
44         selectedImage = imgDown
45     } else {
46         selectedImage = img
47     }
48
49     g.drawTransformedPicture(posX, posY, this.angle, 0.1f, selectedImage)
50     g.drawTransformedPicture(posX, posY, this.angle / 2, 0.1f, rider)
51
52 }
53
54 def loadImages(): Unit = {
55     if (firstRun) {
56         if (img == null) {
57             rider = new BitmapImage("./icons/no-sled-mud.png")
58             img = new BitmapImage("./icons/sledge.png")
59             imgDown = new BitmapImage("./icons/sledge.png")
60             selectedImage = img
61         }
62         firstRun = false
63     }
64 }
65
66 def update(): Unit = {
67     posX = this.getBodyPosition.x
68     posY = this.getBodyPosition.y
69 }
70
71 /** Called for every collision. */
72 override def collision(other: AbstractPhysicsObject, energy: Float): Unit = {
73     //this.angle = this.angle + 1 //prout <----- (commentaire relique de St-Mui)
74     angle = (other.getBodyAngle * 180 / math.Pi).toFloat
75
76     if (angle <= -120) {
77         angle += 180
78     }
79     if (angle >= 120) {
80         angle -= 180
81     }
82
83     lastCollision = 1.0f
84 }
85 }

```

Listing 4 - Personnage physique du jeu - MudryMachine.scala

```

1 import java.io.File
2 import javax.sound.sampled.AudioSystem
3
4 // THIS CLASS IS RESPONSIBLE FOR MUSIC PART OF THE GAME
5
6 class MusicPlayer() {
7     private val clip = AudioSystem.getClip()
8     private var lastmode: String = ""
9     private var lastMusicmode: String = ""
10
11     def play(currentMode: String, musicMode: String): Unit = {
12         if (musicMode == "musicmute" && clip.isActive) { // Si la musique est coupée en jeu, fermer le clip
13             lastMusicmode = "musicmute"
14             if (clip.isOpen) {
15                 clip.stop()
16                 clip.close()
17             }
18         }
19
20         if (currentMode != lastmode && musicMode != "musicmute" ||

```

```

21 lastMusicmode != musicMode && musicMode != "musicmute" ||
22 !clip.isActive && musicMode != "musicmute") {
23 currentMode match { // Else, we play the music based on the game mode
24 case "play" => {
25     if (!clip.isActive || lastmode != "play") { //if the mode play is selected, play the music "play.wav"
26         if (lastmode != currentMode) {
27             if (clip.isOpen){
28                 clip.stop()
29                 clip.close()
30             }
31             lastmode = currentMode
32         }
33
34         val musicfile = new File(s"./music/play.wav")
35         val audio = AudioSystem.getAudioInputStream(musicfile)
36         clip.open(audio)
37         clip.start()
38     }
39 }
40 case ("free" | "lines" | "eraser" | "mop") => { //if an edition mode is selected, play the music "edit.wav"
41     if (lastmode != "free" && lastmode != "lines" && lastmode != "mop" && lastmode != "eraser") {
42         if (clip.isOpen){
43             clip.stop()
44             clip.close()
45         }
46     }
47     if (!clip.isActive &&
48         (lastmode != "free" || lastmode != "lines" || lastmode != "mop" || lastmode != "eraser")) {
49         if (lastmode != currentMode) {
50             if (clip.isOpen){
51                 clip.stop()
52                 clip.close()
53             }
54             lastmode = currentMode
55         }
56
57         val musicfile = new File(s"./music/edit.wav")
58         val audio = AudioSystem.getAudioInputStream(musicfile)
59         clip.open(audio)
60         clip.loop(-1)
61         clip.start()
62     }
63 }
64 case _ => { //else stop everything
65     lastmode = currentMode
66     if (clip.isOpen){
67         clip.stop()
68         clip.close()
69     }
70 }
71 }
72 }
73 lastmode = currentMode
74 }
75 }

```

Listing 5 - Gestion de la musique - MusicPlayer.scala

9.3 – Systèmes de dessin

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3 import com.badlogic.gdx.math.Vector2
4
5 import java.io.{FileOutputStream, PrintWriter}
6 import scala.collection.mutable.ArrayBuffer
7 import scala.io.Source
8
9 // THIS CLASS IS RESPONSIBLE FOR DRAWING THE LINES, EITHER DECO OR PHYSICS

```

```

10 // IT ALSO CAN REMOVE LINES (MOP, ERASER)
11 // IT SAVES THE LINES IN THE SAVE
12 // IT LOADS ANY LINES FROM A SAVE
13
14
15 class LineDrawMachine {
16     val calc: Calculator = new Calculator
17     var LineArray: ArrayBuffer[Line] = ArrayBuffer.empty
18     var startPoint: Vector2 = new Vector2()
19     var endPoint: Vector2 = new Vector2()
20     var lastEndPoint: Vector2 = new Vector2()
21     var isMousePressed: Boolean = false
22     var firstRun: Boolean = true
23     var cursorLoc = new Vector2()
24
25     ArrayEmptyFix()
26
27     // IF THE LINE IS CLOSE TO THE VISIBLE AREA OF THE CAMERA, IT DRAWS IT
28     def drawLines(g: GdxGraphics, cm: String, dm: String, camX: Int, camY: Int): Unit = {
29         if (dm != "decoration") g.setColor(Color.BLACK) else g.setColor(Color.BLUE)
30         if (endPoint.x != 0.0f && endPoint.y != 0.0f) g.drawLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
31         for (line <- LineArray) {
32             if (line.isInstanceOf[DecoLine] && cm != "play") {
33                 line.color = Color.BLUE
34             } else {
35                 line.color = Color.BLACK
36             }
37             if (line.p1x >= (camX-2000) && line.p1x <= (camX+2000) ||
38                 line.p2x >= (camX-2000) && line.p2x <= (camX+2000) ||
39                 line.p1y >= (camY-1500) && line.p1y <= (camY+1500) ||
40                 line.p2y >= (camY-1500) && line.p2y <= (camY+1500) ||
41                 line.p1x == -10000 && line.p2y == -10000) {
42                 line.draw(g)
43             }
44         }
45     }
46
47     def onClick(mode: String, x: Int, y: Int): Unit = {
48         mode match {
49             case "RIGHT" => {
50
51             }
52             case "LEFT" => {
53                 if (lastEndPoint.x == endPoint.x && lastEndPoint.y == endPoint.y) endPoint.set(x, y)
54                 startPoint.set(x, y)
55                 isMousePressed = true
56             }
57         }
58     }
59
60     def onDrag(x: Int, y: Int): Unit = {
61         endPoint.set(x, y)
62     }
63
64     def onRelease(mode: String, x: Int, y: Int, dm: String): Unit = {
65         mode match {
66             case "RIGHT" => {
67
68             }
69             case "LEFT" => {
70                 isMousePressed = false
71                 endPoint.set(x, y)
72
73                 dm match {
74                     case "physic" => {
75                         val l1 = PhysicLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
76                         LineArray.addOne(l1)
77                         lastEndPoint.set(endPoint.x, endPoint.y)
78                     }
79                     case "decoration" => {

```



```

80     val l1 = DecoLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
81     LineArray.addOne(l1)
82     lastEndPoint.set(endPoint.x, endPoint.y)
83   }
84 }
85 }
86 }
87 }
88
89 def clean(x: Int, y: Int): Unit = {
90   ArrayEmptyFix()
91   cursorLoc.set(x, y)
92   val toRemove: ArrayBuffer[Line] = ArrayBuffer.empty
93
94   for (line <- LineArray) {
95     if (calc.isPointInLine(line, cursorLoc)) {
96       if (!toRemove.contains(line))
97         toRemove.addOne(line)
98     }
99   }
100   for (line <- toRemove) {
101     LineArray -= line
102     line.destroy()
103   }
104   endPoint.set(0f, 0f)
105   startPoint.set(0f, 0f)
106   ArrayEmptyFix()
107 }
108
109 def mop(): Unit = {
110   for (line <- LineArray) {
111     line.destroy()
112   }
113   LineArray.clear()
114   ArrayEmptyFix()
115 }
116
117 private def ArrayEmptyFix(): Unit = {
118   if (LineArray.isEmpty) {
119     val l1 = PhysicLine(-10000, -10000, -10000, -10000)
120     LineArray.addOne(l1)
121   }
122 }
123
124 def save(filename: String): Unit = {
125   val pw = new PrintWriter(
126     new FileOutputStream(s"./saves/$filename.csv", true)
127   )
128   for (line <- LineArray) {
129     pw.println(line.getClass.getSimpleName + ";" +
130       line.p1x + ";" + line.p1y + ";" + line.p2x + ";" + line.p2y)
131   }
132   pw.close()
133 }
134
135 def load(fp: String): Unit = {
136   val src = Source.fromFile(s"./saves/$fp")
137   val lines = src.getLines().toArray
138   for (line <- lines) {
139     val a = line.split(",")
140     if (line.contains("PhysicLine")) {
141       LineArray.addOne(PhysicLine(a(1).toFloat, a(2).toFloat, a(3).toFloat, a(4).toFloat))
142     }
143     if (line.contains("DecoLine")) {
144       LineArray.addOne(DecoLine(a(1).toFloat, a(2).toFloat, a(3).toFloat, a(4).toFloat))
145     }
146   }
147   src.close()
148 }
149 }

```

Listing 6 - Dessin de lignes segmentées - LineDrawMachine.scala

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3 import com.badlogic.gdx.math.Vector2
4 import typesLibrary.Free
5
6 import java.io.{FileOutputStream, PrintWriter}
7 import scala.collection.mutable.ArrayBuffer
8
9 // THIS CLASS IS RESPONSIBLE FOR DRAWING THE FREE DRAWINGS, EITHER DECO OR PHYSICS
10 // IT ALSO CAN REMOVE LINES (MOP, ERASER)
11 // FINALLY IT SAVES THE FREELINES IN THE SAVE
12
13 class FreeDrawMachine {
14   val calc: Calculator = new Calculator
15   var FreeArray: ArrayBuffer[Free] = ArrayBuffer.empty
16   var startPoint: Vector2 = new Vector2()
17   var endPoint: Vector2 = new Vector2()
18   var lastEndPoint: Vector2 = new Vector2()
19   var isMousePressed: Boolean = false
20   var FreeCnt: Int = -1
21   var firstRun: Boolean = true
22   var cursorLoc = new Vector2()
23
24   ArrayEmptyFix()
25
26   // IF THE LINE IS CLOSE TO THE VISIBLE AREA OF THE CAMERA, IT DRAWS IT
27   def drawFreeLines(g: GdxGraphics, cm: String, dm: String, camX: Int, camY: Int): Unit = {
28     if (dm != "decoration") g.setColor(Color.BLACK) else g.setColor(Color.BLUE)
29     if (endPoint.x != 0.0f && endPoint.y != 0.0f) g.drawLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
30     for (free <- FreeArray) {
31       for (segment <- free) {
32         if (segment.isInstanceOf[DecoLine] && cm != "play") {
33           segment.color = Color.BLUE
34         } else {
35           segment.color = Color.BLACK
36         }
37         if (segment.p1x >= (camX-2000) && segment.p1x <= (camX+2000) ||
38             segment.p2x >= (camX-2000) && segment.p2x <= (camX+2000) ||
39             segment.p1y >= (camY-1500) && segment.p1y <= (camY+1500) ||
40             segment.p2y >= (camY-1500) && segment.p2y <= (camY+1500) ||
41             segment.p1x == -10000 && segment.p2y == -10000) {
42           segment.draw(g)
43         }
44       }
45     }
46   }
47 }
48
49 def onClick(mode: String, x: Int, y: Int): Unit = {
50   mode match {
51     case "RIGHT" => {
52       }
53     case "LEFT" => {
54       val f1 = new Free
55       FreeArray.addOne(f1)
56       FreeCnt += 1
57       if (lastEndPoint.x == endPoint.x && lastEndPoint.y == endPoint.y) endPoint.set(x, y)
58       startPoint.set(x, y)
59       isMousePressed = true
60     }
61   }
62 }
63
64 def onDrag(x: Int, y: Int, dm: String): Unit = {
65   endPoint.set(x, y)
66   dm match {
67     case "physic" => {

```

```

69     val seg1 = PhysicLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
70     lastEndPoint.set(endPoint.x, endPoint.y)
71     FreeArray(FreeCnt).addOne(seg1)
72     startPoint.set(endPoint.x, endPoint.y)
73 }
74 case "decoration" => {
75     val seg1 = DecoLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
76     lastEndPoint.set(endPoint.x, endPoint.y)
77     FreeArray(FreeCnt).addOne(seg1)
78     startPoint.set(endPoint.x, endPoint.y)
79 }
80 }
81 }
82
83 def onRelease(mode: String, x: Int, y: Int): Unit = {
84     mode match {
85         case "RIGHT" => {
86
87         }
88         case "LEFT" => {
89             isMousePressed = false
90
91         }
92     }
93 }
94
95 def clean(x: Int, y: Int): Unit = {
96     ArrayEmptyFix()
97     cursorLoc.set(x, y)
98     val toRemove: ArrayBuffer[Line] = ArrayBuffer.empty
99
100     for (free <- FreeArray) {
101         for (segment <- free) {
102             if (calc.isPointInSegment(segment, cursorLoc)) {
103                 if (!toRemove.contains(segment))
104                     toRemove.addOne(segment)
105             }
106         }
107     }
108     for (free <- FreeArray) {
109         for (segment <- toRemove) {
110             if (free.contains(segment)) {
111                 free -= segment
112                 segment.destroy()
113             }
114         }
115     }
116 }
117
118 def mop(): Unit = {
119     for (free <- FreeArray) {
120         for (segmt <- free) {
121             segmt.destroy()
122         }
123     }
124     for (free <- FreeArray) {
125         free.clear()
126     }
127     ArrayEmptyFix()
128 }
129
130 private def ArrayEmptyFix(): Unit = {
131     if (FreeArray.isEmpty) {
132         val l1 = PhysicLine(-10000, -10000, -10000, -10000)
133         FreeArray.addOne(new ArrayBuffer[Line]().addOne(l1))
134     }
135 }
136
137 def save(filename: String): Unit = {
138     val pw = new PrintWriter(

```

```

139     new FileOutputStream(s"./saves/$filename.csv", true)
140   )
141   for (free <- FreeArray) {
142     for (sgmt <- free) {
143       pw.println(sgmt.getClass.getSimpleName + "," +
144         sgmt.p1x + "," + sgmt.p1y + "," + sgmt.p2x + "," + sgmt.p2y)
145     }
146   }
147   pw.close()
148 }
149
150 }

```

Listing 7 - Dessin de lignes libres - FreeDrawMachine.scala

```

1  import com.badlogic.gdx.math.Vector2
2
3  // THIS CLASS IS RESPONSIBLE FOR THE CALCULATION
4  // THIS LOGIC IS USED FOR THE ERASER FUNCTION
5
6  class Calculator {
7    val tolerance = 8 //Mouse tolerance, nbr of pixels around the mouse
8    var vDir = new Vector2() //Vecteur Directeur
9    var vXP = new Vector2() // Vecteur entre le point X et le point P (P = Mouse location)
10
11    //The logic is the same as a Segment, but to make it consistant, we added this fonction that equals isPointInSegment
12    def isPointInLine(sgmt: Line, point: Vector2): Boolean = {
13      isPointInSegment(sgmt, point)
14    }
15
16    //The mouse is in the segment : if the distance between it is in tolerance and if it is on the said segment
17    def isPointInSegment(sgmt: Line, point: Vector2): Boolean = {
18      if (distanceToSegment(sgmt, point) <= tolerance) {
19        if (isPointInBoundaries(sgmt, point)) {
20          return true
21        }
22      }
23      false
24    }
25
26    //Returns true if the point is part of a said segment
27    def isPointInBoundaries(sgmt: Line, point: Vector2): Boolean = {
28      val minX = math.min(sgmt.p1x, sgmt.p2x) - tolerance
29      val maxX = math.max(sgmt.p1x, sgmt.p2x) + tolerance
30      val minY = math.min(sgmt.p1y, sgmt.p2y) - tolerance
31      val maxY = math.max(sgmt.p1y, sgmt.p2y) + tolerance
32
33      if (minX <= point.x && point.x <= maxX) {
34        if (minY <= point.y && point.y <= maxY) {
35          return true
36        }
37      }
38      false
39    }
40
41    //Returns the distance between the mouse and the vector
42    def distanceToSegment(sgmt: Line, point: Vector2): Double = {
43      vDir.set((sgmt.p2x - sgmt.p1x), (sgmt.p2y - sgmt.p1y))
44      vXP.set((point.x - sgmt.p1x), (point.y - sgmt.p1y))
45      val vectorProduct = vDir.x * vXP.y - vDir.y * vXP.x
46      var vDirNorm = math.sqrt(vDir.x * vDir.x + vDir.y * vDir.y)
47      if (vDirNorm == 0) vDirNorm = Double.MinPositiveValue //Removing the issue of dividing by zero
48      val distance = (math.abs(vectorProduct) / vDirNorm)
49      distance
50    }
51  }

```

Listing 8 - Calculs géométriques - Calculator.scala

9.4 – Types de lignes

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3
4 trait Line {
5   var color: Color
6
7   def p1x: Float
8
9   def p1y: Float
10
11  def p2x: Float
12
13  def p2y: Float
14
15  def destroy() {}
16
17  def draw(g: GdxGraphics) {}
18
19 }

```

Listing 9 - Trait de base pour les lignes - Line.scala

```

1 import ch.hevs.gdx2d.components.physics.primitives.PhysicsStaticLine
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import ch.hevs.gdx2d.lib.physics.AbstractPhysicsObject
4 import com.badlogic.gdx.graphics.Color
5 import com.badlogic.gdx.math.Vector2
6
7 case class PhysicLine(p1x: Float, p1y: Float, p2x: Float, p2y: Float)
8 extends PhysicsStaticLine("a line", new Vector2(p1x, p1y), new Vector2(p2x, p2y), 0.1f, 0.5f, 0.4f) with Line {
9   var color: Color = Color.BLACK
10
11  override def collision(other: AbstractPhysicsObject, energy: Float): Unit = {
12  }
13
14  override def destroy(): Unit = {
15    super[PhysicsStaticLine].destroy()
16  }
17
18  override def draw(g: GdxGraphics): Unit = {
19    g.setColor(color)
20    g.drawLine(p1x, p1y, p2x, p2y)
21  }
22 }

```

Listing 10 - Ligne physique - PhysicLine.scala

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3
4 // THIS CASE CLASS IS STORING ALL THE DATA ABOUT A SAID DECOLINE
5
6 case class DecoLine(p1x: Float, p1y: Float, p2x: Float, p2y: Float)
7 extends Line {
8   var color: Color = Color.BLACK;
9
10  override def destroy() = {
11    super.destroy()
12  }
13
14  override def draw(g: GdxGraphics): Unit = {
15    g.setColor(color)
16    g.drawLine(p1x, p1y, p2x, p2y)
17  }
18 }

```

Listing 11 - Ligne de décoration - DecoLine.scala

9.5 – Fenêtres UI

```

1 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2 import ch.hevs.gdx2d.components.screen_management.RenderingScreen
3 import ch.hevs.gdx2d.lib.GdxGraphics
4 import ch.hevs.gdx2d.lib.physics.PhysicsWorld
5 import com.badlogic.gdx.Gdx
6 import com.badlogic.gdx.graphics.Color
7 import com.badlogic.gdx.graphics.g2d.BitmapFont
8 import com.badlogic.gdx.graphics.g2d.freetype.FreeTypeFontGenerator
9 import com.badlogic.gdx.graphics.g2d.freetype.FreeTypeFontGenerator.FreeTypeFontParameter
10 import com.badlogic.gdx.math.Vector2
11
12
13 class SplashScreenWindow extends RenderingScreen {
14
15     private val param: FreeTypeFontParameter = new FreeTypeFontParameter
16     param.size = 50
17     param.color = Color.BLACK
18     private val customFont: BitmapFont =
19         new FreeTypeFontGenerator(Gdx.files.local("font/Acme-Regular.ttf")).generateFont(param)
20     var mM: MudryMachine = _
21     var c = 0
22     private var imgBitmap: BitmapImage = _
23
24     override def onInit(): Unit = {
25         imgBitmap = new BitmapImage("icons/mudry2.png")
26         mM = new MudryMachine("rider2", new Vector2(960, 900), 10, 1f, 0.65f, 0.6f)
27         mM.loadImages()
28         mM.setPos(100, 1200)
29     }
30
31     override def onGraphicRender(g: GdxGraphics): Unit = {
32         g.clear(Color.WHITE)
33         g.setColor(Color.BLACK)
34         g.drawStringCentered(((g.getScreenHeight / 2) + c).toFloat, "MudRYder", customFont)
35         c += 2
36         g.setColor(Color.WHITE)
37
38         mM.awake()
39         mM.update()
40         PhysicsWorld.updatePhysics()
41
42         g.drawTransformedPicture(mM.posX, mM.posY, 0, 0.7f, imgBitmap)
43         g.drawSchoolLogo()
44     }
45
46     override def dispose(): Unit = {
47         imgBitmap.dispose()
48     }
49
50 }

```

Listing 12 - Écran de démarrage - SplashScreenWindow.scala

```

1 import ch.hevs.gdx2d.components.screen_management.RenderingScreen
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.Gdx
4 import com.badlogic.gdx.files.FileHandle
5 import com.badlogic.gdx.graphics.Color
6 import com.badlogic.gdx.math.Vector2
7
8 import scala.collection.mutable.ListBuffer
9
10
11 class LoadWindow extends RenderingScreen {
12     var files: Array[FileHandle] = _
13     var filesList: ListBuffer[String] = ListBuffer.empty
14
15     var cancelButtonCo: Vector2 = new Vector2(0, 0)

```

```

16 var loadButtonCo: Vector2 = new Vector2(0, 0)
17 var answer: String = ""
18 var selectedSaveIndice: Int = -1
19
20 override def onInit(): Unit = {
21   files = Gdx.files.local("saves").list()
22   files.foreach { f =>
23     if(f.name().contains(".csv"))
24       filesList addOne(f.name())
25   }
26 }
27
28 override def onGraphicRender(g: GdxGraphics): Unit = {
29   g.clear(Color.WHITE)
30   g.setColor(Color.BLACK)
31   g.drawStringCentered(800, "Files available : ")
32   var c = 40
33
34   for (filename <- filesList) {
35     g.drawStringCentered(800 - c, filename)
36
37     if (selectedSaveIndice != -1) {
38       if (filename == filesList(selectedSaveIndice)) {
39         g.setColor(Color.BLUE)
40         g.drawRectangle(960, (795 - c), 160, 20, 0)
41         g.setColor(Color.BLACK)
42       }
43     }
44
45     c += 30
46   }
47
48   if (cancelButtonCo.x == 0 && cancelButtonCo.y == 0) {
49     cancelButtonCo = new Vector2((g.getScreenWidth / 2 - 80), (750 - c))
50   }
51   if (loadButtonCo.x == 0 && loadButtonCo.y == 0) {
52     loadButtonCo = new Vector2((g.getScreenWidth / 2 + 80), (750 - c))
53   }
54
55   g.drawRectangle(cancelButtonCo.x, cancelButtonCo.y, 100, 30, 0)
56   g.drawString((g.getScreenWidth / 2 - 105), 755 - c, "Cancel")
57   g.drawRectangle(loadButtonCo.x, loadButtonCo.y, 100, 30, 0)
58   g.drawString((g.getScreenWidth / 2 + 65), 755 - c, "Load")
59 }
60
61 def onClick(x: Int, y: Int): Unit = {
62   if (x >= cancelButtonCo.x - 50 && x <= cancelButtonCo.x + 50 &&
63       y >= cancelButtonCo.y - 15 && y <= cancelButtonCo.y + 15) {
64     answer = "cancel" // Cancel
65   } else if (x >= loadButtonCo.x - 50 && x <= loadButtonCo.x + 50 &&
66             y >= loadButtonCo.y - 15 && y <= loadButtonCo.y + 15) {
67     answer = "load" // Load
68   } else {
69     answer = ""
70   }
71 }
72
73 for (c <- 0 until (filesList.length * 30) by 30) {
74   if (x >= 880 && x <= 1040 &&
75       y >= 745 - c && y <= 765 - c) {
76     selectedSaveIndice = c / 30
77   }
78 }
79
80 }
81
82 def getSelection(): String = {
83   filesList(selectedSaveIndice)
84 }
85
86

```

```

87 def getAnswer(): String = {
88     answer
89 }
90
91 }

```

Listing 13 - Fenêtre de chargement - LoadWindow.scala

```

1 import ch.hevs.gdx2d.components.graphics.Polygon
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.graphics.Color
4 import com.badlogic.gdx.math.Vector2
5
6 class SavingWindow {
7
8     val firstWindowPoint: Vector2 = new Vector2(280, 960)
9     val windowHeight: Int = 55
10    val windowWidth: Int = 165
11
12    val windowCo: Array[Vector2] = Array(
13        firstWindowPoint,
14        new Vector2(firstWindowPoint.x, firstWindowPoint.y + windowHeight),
15        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y + windowHeight),
16        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y)
17    )
18
19    def drawWindow(g: GdxGraphics, fn: String): Unit = {
20        val oldColor = g.sbGetColor()
21        g.setColor(Color.BLACK)
22        g.drawFilledPolygon(new Polygon(windowCo), Color.WHITE)
23        g.drawPolygon(new Polygon(windowCo))
24        g.drawString(firstWindowPoint.x + 33, firstWindowPoint.y + windowHeight - 10, "Saving File...")
25        g.drawString(firstWindowPoint.x + 10, firstWindowPoint.y + windowHeight - 33, fn)
26        g.setColor(oldColor)
27    }
28
29 }

```

Listing 14 - Notification de sauvegarde - SavingWindow.scala

```

1 import ch.hevs.gdx2d.components.graphics.Polygon
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.graphics.Color
4 import com.badlogic.gdx.math.Vector2
5
6 // THIS CLASS IS RESPONSIBLE FOR THE SMALL WINDOW SHOWING IN CASE OF MOPPING
7
8 class AreYouSureWindow {
9     //Window Size
10    val firstWindowPoint: Vector2 = new Vector2(200, 960)
11    val windowHeight: Int = 60
12    val windowWidth: Int = 130
13
14    //Window Coordinates
15    val windowCo: Array[Vector2] = Array(
16        firstWindowPoint,
17        new Vector2(firstWindowPoint.x, firstWindowPoint.y + windowHeight),
18        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y + windowHeight),
19        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y)
20    )
21
22    //Button Margins
23    val buttonMargin: Int = 10
24    val buttonHeight: Int = 20
25    val buttonWidth: Int = 50
26
27    //Yes Button Coordinates
28    val firstButtonPoint: Vector2 = new Vector2(firstWindowPoint.x + buttonMargin, firstWindowPoint.y + buttonMargin)

```



```

29 val yesButton: Array[Vector2] = Array(
30     firstButtonPoint,
31     new Vector2(firstButtonPoint.x, firstButtonPoint.y + buttonHeight),
32     new Vector2(firstButtonPoint.x + buttonWidth, firstButtonPoint.y + buttonHeight),
33     new Vector2(firstButtonPoint.x + buttonWidth, firstButtonPoint.y)
34 )
35 //No Button Coordinates
36 val noButtonPoint: Vector2 = new Vector2(firstButtonPoint.x + buttonWidth + buttonMargin, firstButtonPoint.y)
37 val noButton: Array[Vector2] = Array(
38     noButtonPoint,
39     new Vector2(noButtonPoint.x, noButtonPoint.y + buttonHeight),
40     new Vector2(noButtonPoint.x + buttonWidth, noButtonPoint.y + buttonHeight),
41     new Vector2(noButtonPoint.x + buttonWidth, noButtonPoint.y)
42 )
43
44 private var answer: String = "" //Answer storage var
45
46 def drawWindow(g: GdxGraphics): Unit = {
47     //Saving old color setting
48     val oldColor = g.sbGetColor()
49
50     //Drawing the window
51     g.setColor(Color.BLACK)
52     g.drawFilledPolygon(new Polygon(windowCo), Color.WHITE)
53     g.drawPolygon(new Polygon(windowCo))
54     g.drawString(firstWindowPoint.x + 13, firstWindowPoint.y + windowHeight - buttonMargin, "Are you sure ?")
55     g.drawPolygon(new Polygon(yesButton))
56     g.drawString(firstButtonPoint.x + 12, firstButtonPoint.y + buttonHeight - 5, "Yes")
57     g.drawPolygon(new Polygon(noButton))
58     g.drawString(noButtonPoint.x + 15, noButtonPoint.y + buttonHeight - 5, "No")
59
60     //Switching back to old color
61     g.setColor(oldColor)
62 }
63
64 def onClick(x: Int, y: Int): Unit = {
65     //If the mouse is inside of the Yes button coordinates, answer = yes
66     if (x >= firstButtonPoint.x && x <= firstButtonPoint.x + buttonWidth &&
67         y >= firstButtonPoint.y && y <= firstButtonPoint.y + buttonHeight) {
68         answer = "yes"
69
70         //If the mouse is inside of the NO button coordinates, answer = no
71     } else if (x >= noButtonPoint.x && x <= noButtonPoint.x + buttonWidth &&
72         y >= noButtonPoint.y && y <= noButtonPoint.y + buttonHeight) {
73         answer = "no" // No
74     } else {
75         answer = ""
76     }
77 }
78
79 def getAnswer(): String = {
80     return answer
81 }
82 }

```

Listing 15 - Fenêtre de confirmation - AreYouSureWindow.scala

9.6 – Modes de jeu

```

1 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.graphics.Color
4
5 import scala.collection.mutable.ArrayBuffer
6
7 class MenuModesMachine {
8     val modes: ArrayBuffer[MenuModes] = ArrayBuffer.empty
9     val icons: ArrayBuffer[BitmapImage] = ArrayBuffer.empty
10    private var cm: String = "lines"

```

```

11 private var dm: String = "physic"
12 private var mm: String = "music"
13 loadModes()
14 private var firstRun: Boolean = true
15
16 def loadIcons(): Unit = {
17   if (firstRun) {
18     for (mode <- modes) {
19       val img = new BitmapImage(s"./icons/${mode.name}.png")
20       icons.addOne(img)
21     }
22     firstRun = false
23   }
24 }
25
26 def drawModesMenu(g: GdxGraphics, width: Int, height: Int): Unit = {
27   val startPointH: Int = width - 20
28   var startPointW: Int = height + 30
29   val radius = 18
30
31   for (i <- modes.indices) {
32     if (modes(i).name == "music" || modes(i).name == "musicmute") {
33       if (getMusicMode() == modes(i).name) {
34         modes(i).x = startPointW
35         modes(i).y = startPointH
36         modes(i).radius = radius
37
38         if (getMusicMode() == modes(i).name) {
39           g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
40         }
41         g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
42         startPointW += 50
43       }
44     } else {
45       if (modes(i).name == "physic" || modes(i).name == "decoration") {
46         if (getDrawMode() == modes(i).name) {
47           modes(i).x = startPointW
48           modes(i).y = startPointH
49           modes(i).radius = radius
50
51           if (getDrawMode() == modes(i).name) {
52             g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
53           }
54           g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
55           startPointW += 50
56         }
57       } else {
58         modes(i).x = startPointW
59         modes(i).y = startPointH
60         modes(i).radius = radius
61
62         if (currentMode() == modes(i).name) {
63           g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
64         }
65         if (getDrawMode() == modes(i).name) {
66           g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
67         }
68         g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
69         startPointW += 50
70       }
71     }
72   }
73 }
74
75 def currentMode(): String = {
76   return cm
77 }
78
79 def getDrawMode(): String = {
80   return dm

```

```

81 }
82
83 def getMusicMode(): String = {
84     return mm
85 }
86
87 def onMenuClick(x: Int, y: Int): Boolean = {
88
89     //vérifier pythagore vu que boutons ronds
90     // Return true si un bouton du menu a été touché (pour prevent les actions quand clic sur le menu)
91     for (m <- modes) {
92         if (math.sqrt((x - m.x) * (x - m.x) + (y - m.y) * (y - m.y)) < m.radius) {
93             // bouton touché -> Changement de mode
94             modeSwitcher(m.name)
95             return true
96         }
97     }
98     false
99 }
100
101 def modeSwitcher(m: String): Unit = {
102     if (m == "physic") {
103         if (dm == "physic") {
104             dm = "decoration"
105         } else {
106             dm = "physic"
107         }
108     } else if (m == "music") {
109         if (mm == "music") {
110             mm = "musicmute"
111         } else {
112             mm = "music"
113         }
114     } else {
115         cm = m
116     }
117 }
118
119 private def loadModes(): Unit = {
120     modes.addOne(MenuModes("free", 0, 0, 10))
121     modes.addOne(MenuModes("lines", 0, 0, 10))
122     modes.addOne(MenuModes("play", 0, 0, 10))
123     modes.addOne(MenuModes("eraser", 0, 0, 10))
124     modes.addOne(MenuModes("mop", 0, 0, 10))
125     modes.addOne(MenuModes("return", 0, 0, 10))
126     modes.addOne(MenuModes("save", 0, 0, 10))
127     modes.addOne(MenuModes("load", 0, 0, 10))
128     modes.addOne(MenuModes("physic", 0, 0, 10))
129     modes.addOne(MenuModes("decoration", 0, 0, 10))
130     modes.addOne(MenuModes("music", 0, 0, 10))
131     modes.addOne(MenuModes("musicmute", 0, 0, 10))
132 }
133 }

```

Listing 16 - Gestionnaire des modes du menu - MenuModesMachine.scala

```

1 case class MenuModes(name: String, var x: Int, var y: Int, var radius: Int) {
2
3 }

```

Listing 17 - Structure des modes du menu - MenuModes.scala

9.7 – Types

```

1 import scala.collection.mutable.ArrayBuffer
2
3 // THIS OBJECT KEEPS TYPES
4

```

```
5 package object typesLibrary {  
6   type Free = ArrayBuffer[Line] // A free draw is a combination of multiple lines  
7 }
```

Listing 18 - Type alias pour les lignes - typesLibrary.scala

Références

- [1] libGDX, « libGDX - Java Game Development Framework ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://libgdx.com/>
- [2] B. Cadez, « Line Rider ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://www.linerider.com/>
- [3] Box2D, « Box2D - A 2D Physics Engine for Games ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://box2d.org/>